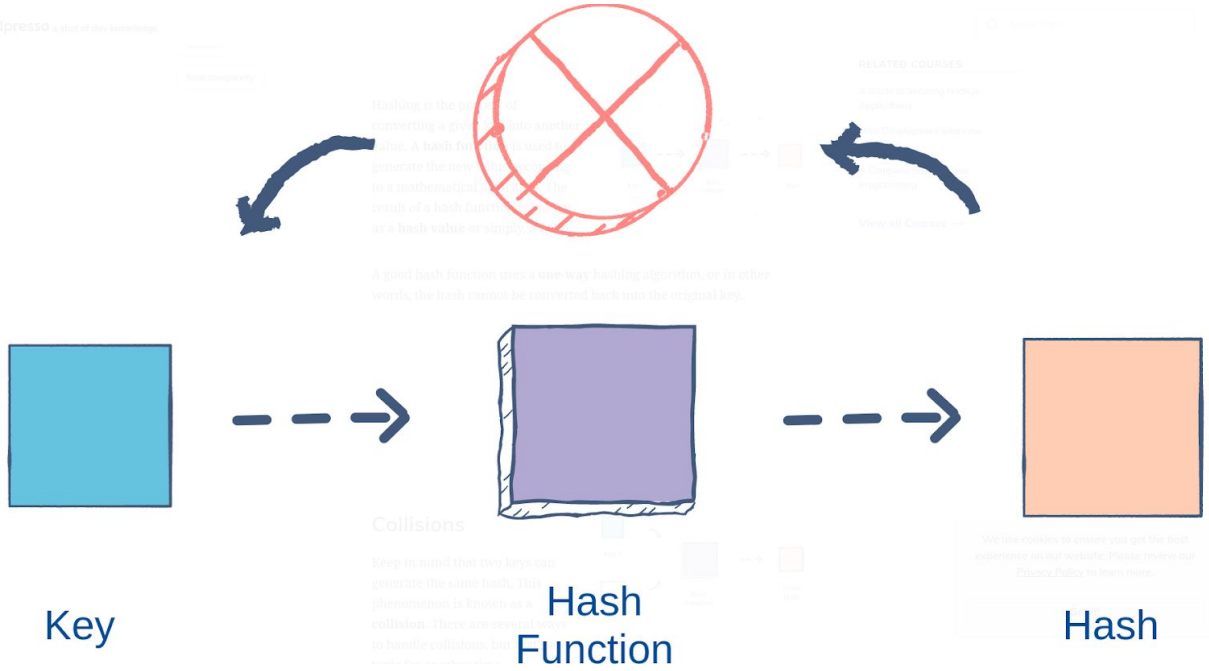
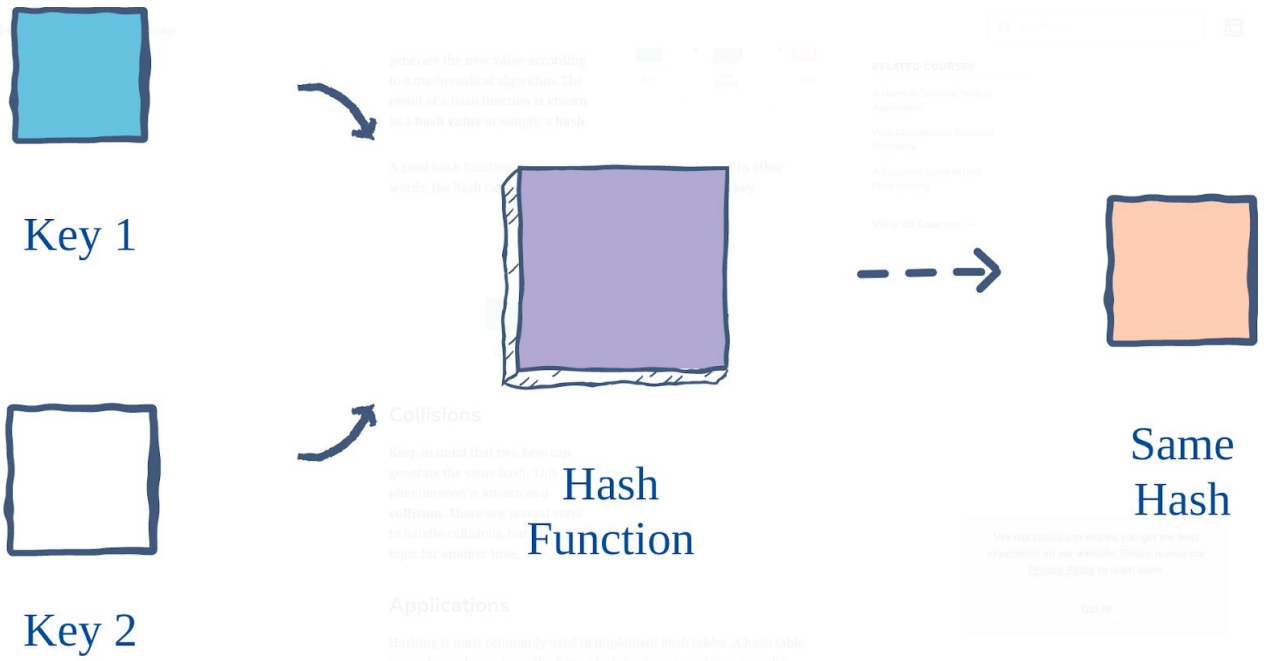
Hash tables

DS 2019/2020

Idea: We have a key and through a hash function we get a hash



2 keys may have the same hash, it's called a collision



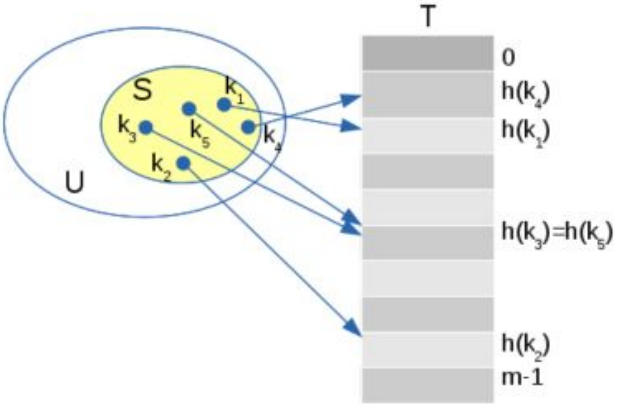
Hash function examples:

- MD5 - obsolete, has many collisions, **DO NOT USE IT EVER!**
- SHA1
- SHA2 - SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, SHA-512/256. SHA-256 and SHA-512
- SHA-3
- and many other underground ones

- U is an universe set, example all the natural numbers
- S is our set, for example an integer vector
- In order to get the position of our element S(k) in T we must calculate h(k) where h is our has function and T the hash table
- h can have collisions, meaning that k1=/=k2, h(k1) = h(k2)

Hash tables

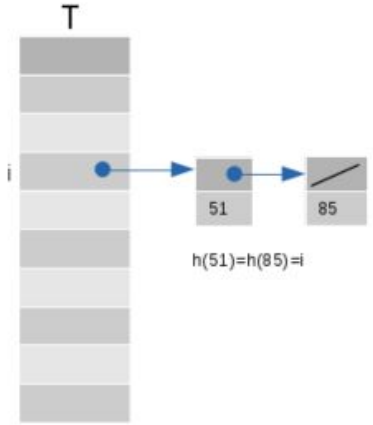
- It uses a hash function h to associate to the keys of universe U a value from the set $\{0, 1, \cdots, m-1\}$.



- An element with the key k has associated the position $h(k)$ in the table T.
- The hash function reduces the domains of indices and implicitly the size of the stored array.
- Collision: $\exists\ x_1, x_2 \in S$ such that $h(x_1.key) = h(x_2.key)$

Collision resolution by chaining

- The records that have associated the same slot will be stored in a linked list. T becomes an array of pointers.



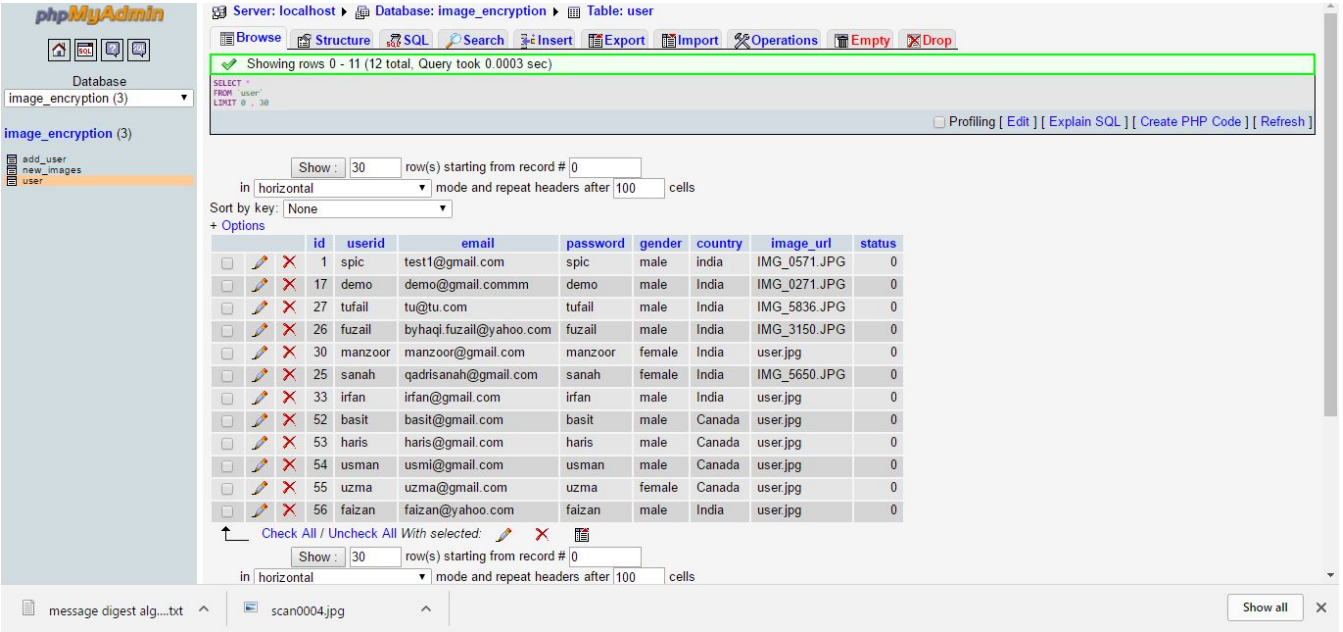
- A simple solution, but it requires additional memory space.
- Worst case scenario: all keys have associated the same slot
  - the access time: $\Theta(n)$.

We will see more collision techniques at the seminar.

Use case

- We are a site.
- We have users.
- Users have passwords.
- Passwords are stored in the database.
- Is it safe to save them in plain text? (meaning just save the database in clear, “in clar”, anybody with access to the database can see the passwords)

Say somebody attacks our database and sees the passwords
21554666d2756b4e4f348999054f68fe3b7c9f02



Now he knows the passwords of all the users.

Many users use the same password everywhere, we are now **liable** under GDPR, we exposed private information.

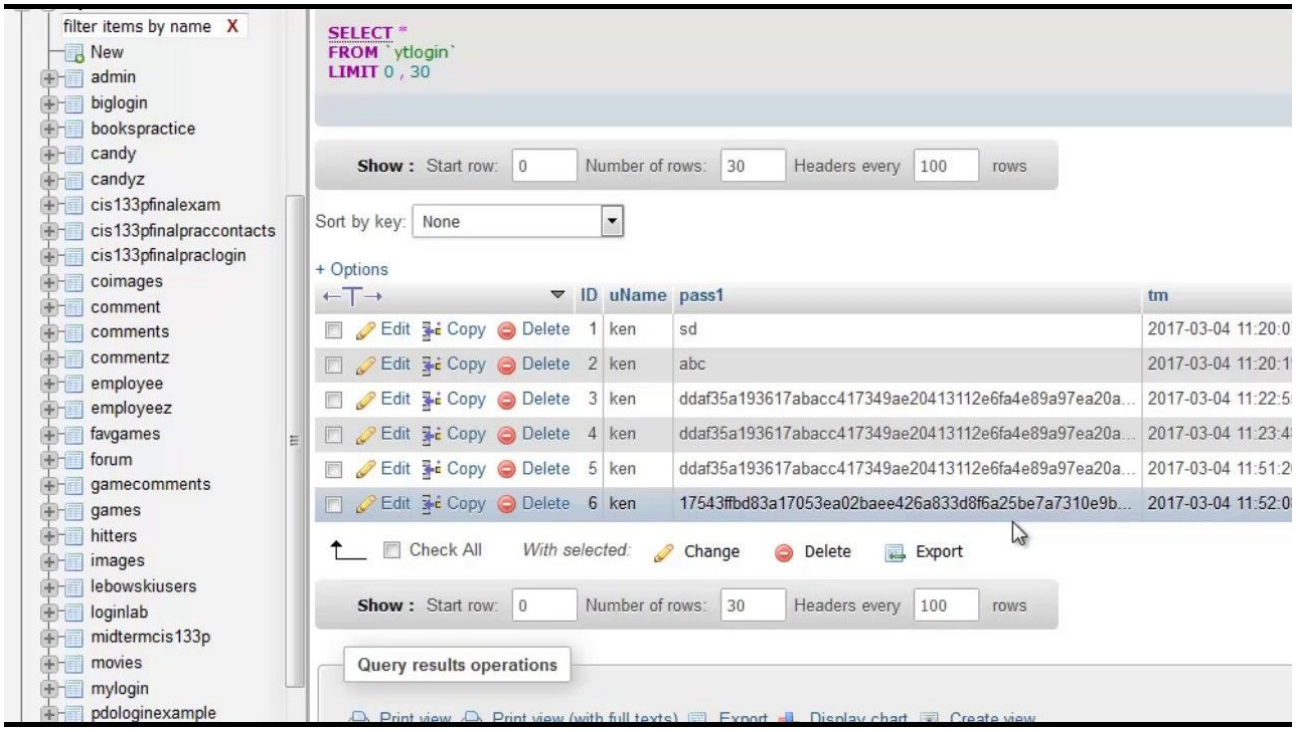
How can we fix this?

We hash the password with a hashing function and store it.

The attacker will access our database but will see the hashed password.

Since the function is one way he cannot find out the original password.

This is what he will see:



There are attacks of hash functions.

If the user chooses a weak password, for example a dictionary word, the attacker can calculate all the hashes and compare with the user hash to guess the password.

Choose strong passwords.

How can we authenticate users?

- Through HTTPS they send the encrypted password in plain text.
- We calculate the hash of the pass
- We compare with what we have in the database
- If they match it's the same person.
- **!!!VERY IMPORTANT, THEY CAN MATCH WITH A COLLISION, THAT IS WHY WE MUST CHOOSE A MODERN HASH FUNCTION!!! DO NOT USE MD5 OR OTHER WEAK ONES!**